Matched-Wait Evaluation of Retrieval-Augmented Transaction Timing under Ethereum EIP-1559: Benchmarking kNN Analogs, Trailing-Rank Heuristics, and 7B Language-Model Planners

Mohammad Pasha[a,*] ; **Mohammed Ali Shaik**[b]

[a] Postdoctoral Research Fellow, CSE Department, SR University, Ananthasagar, Warangal 506371, Telangana, India
[b] Associate Professor, School of Computer Science and Artificial Intelligence, SR University, Ananthasagar, Warangal 506371, Telangana, India

*Corresponding author: Mohammad Pasha <mohd.pasha@outlook.com>; ORCID: 0000-0002-0596-1682 | Mohammed Ali Shaik: <niharali@gmail.com>; ORCID: 0000-0002-5520-0830

Published Date: 2026-10-18

EIP-1559 Ethereum Mainnet Benchmark

PeerJ Computer Science

Reproducible Open-Source Evaluation

ABSTRACT

Background. Execution-timing decisions, such as deferring an Ethereum transaction within a short deadline, present an operational testbed for retrieval-augmented sequential decision making. An autonomous agent can query similar historical market states and act upon subsequent fee dynamics. However, whether analogue retrieval provides alpha beyond simple trailing-price heuristics once policies are evaluated at identical average waiting budgets remains largely untested.

Methods. We evaluated 316800 contiguous Ethereum mainnet blocks (2024-01-01 to 2024-02-14 UTC; BigQuery public datasets) partitioned chronologically into library (LIB), calibration (CAL), and holdout test (TEST) windows. A swap transaction arriving at block to could defer execution up to a deadline of D = 50 blocks (10.0 minutes) and paid the base fee of its submission block plus a fixed 1.5 gwei tip (150000 gas). We benchmarked submit-immediately, random-wait, a trailing percentile-rank heuristic, non-parametric kNN retrieval over causal state features, an ex-post oracle, and an open-weight 7B language model (qwen2.5:7b; Yang et al., 2024) with and without retrieved context. Heuristic and retrieval thresholds were matched to equal mean wait, and uncertainty was quantified using window-clustered bootstrap percentile intervals (3000 resamples).

Results. On the calibration split, retrieved historical neighbors carried genuine predictive signal regarding achievable savings beyond trailing rank (incremental correlation r = +0.227, 95% cluster CI [+0.167, +0.267]). On holdout test windows, unconstrained kNN retrieval saved 5.25% [3.19, 7.31] but expanded wait to 261s, while the heuristic saved 4.61% [3.78, 5.40] at 165s. When deconfounded across the common support frontier (60s to 400s), the mean Pareto frontier gap was $\Delta = -0.14$ pp (95% CI [−0.70, +0.46]). Because the confidence interval half-width (±0.58 pp) exceeds our pre-specified economic equivalence margin of ±0.40 pp, the comparison is statistically inconclusive rather than an established tie. The 7B language model failed to outperform matched-wait baselines across all prompt configurations (retrieval vs. grid-matched heuristic: $\Delta = -1.05$ pp [−2.61, +0.40]) and exhibited severe wording fragility on calibration ablations.

Conclusions. Although historical state retrieval captures non-redundant predictive signal, we found no statistical evidence that it improves the fee–wait frontier over a matched-wait trailing-rank heuristic. Apparent performance gains in unconstrained retrieval and LLM agents are confounded by waiting budget differences.

Keywords: Retrieval-augmented generation, k-nearest neighbors, Ethereum base fee, EIP-1559, transaction timing, large language models, causal deconfounding, matched-wait evaluation

1. Introduction

Decentralized financial protocols frequently require operational decisions under tight execution deadlines where only recent on-chain history

is observable. Following Ethereum's EIP-1559 upgrade, every block sets an algorithmic base fee per gas that adjusts deterministically based on

previous block gas utilization:

$$b_{t+1} = b_t \left(1 + \tfrac{1}{8}\left[(u_t - u_{target}) / u_{target}\right]\right)$$

where $u_{target} = 15 \times 10^6$ gas (50% of the maximum 30×10^6 block gas limit). In plain terms, when a block's gas utilization exceeds this target, the base fee automatically increases by up to 12.5% for the subsequent 12-second block slot; conversely, when utilization is below target, the base fee decreases by up to 12.5%, and remains constant when gas usage exactly equals u_{target}. Transactions that defer submission can wait for local congestion dips, paying a lower base fee. However, waiting incurs latency risk, and base-fee dynamics exhibit non-stationary bursts and mean-reverting regimes.

Retrieval-augmented decision making has emerged as a promising paradigm to enhance sequential policy execution: by identifying similar market states

in historical memory libraries, an agent or language-model planner can condition its timing on empirical outcome distributions. To our knowledge, no peer-reviewed empirical study evaluates retrieval-augmented execution timing under a matched-wait protocol. Nevertheless, two methodological failure modes pervade current evaluations:

1. **Budget Confounding:** Because lower fees correlate mechanically with longer deferrals, policies that wait longer appear superior, confounding market signal with wait duration.
2. **Prediction-Value Wedge (a term introduced in this study):** Demonstrating statistical correlation between retrieved analogs and future prices does not guarantee economic surplus in real-time execution policies—a phenomenon paralleling the "feature-validity-to-policy-utility gap" identified by Yang (2026) in quantitative trading agents.

We address these challenges by formalizing three core research questions:

- **(Q1)** Do retrieved analogue states carry incremental predictive information about achievable fee savings beyond a trailing percentile-rank heuristic?
- **(Q2)** Does a kNN retrieval policy improve the fee–wait Pareto trade-off over a trailing-rank heuristic when deconfounded at matched mean wait?
- **(Q3)** Can an open-weight 7B language model leverage retrieved historical summaries to improve execution decisions, and how robust is its policy to prompt framing?

Importantly, the primary contribution of this work is methodological rather than algorithmic. The retrieval systems and language-model planner are evaluated as candidate timing policies through which the matched-wait evaluation framework is demonstrated. The central contribution is therefore the causal evaluation methodology and associated deconfounding protocol, rather than the introduction of a novel retrieval architecture or language model.

1.1 Scope as a Minimal Viability Experiment

To rigorously evaluate whether language models provide utility in transaction execution, we formulate the LLM arm as a Minimal Viability Experiment (MVE), a term introduced in this study, for the compound hypothesis (H) that language models improve blockchain fee optimization. This compound hypothesis relies on three necessary conditions in strict causal sequence ($H_1 \rightarrow H_2 \rightarrow H_3$):

1. **Contextual Extraction (H_1):** A language model can extract predictive signals from raw, unstructured on-chain data (such as pending mempool transactions, gas oracle logs, or event traces).
2. **Signal Consumption (H_2):** A language model can ingest a structured predictive signal and translate it into a calibrated decision rule.
3. **Policy Superiority (H_3):** Execution policies driven by the model's signal use outperform parameter-matched domain heuristics on cost and latency objectives.

Because contextual extraction operates strictly upstream of signal consumption and policy execution, downstream failure renders upstream capabilities economically inert. If an agent cannot convert a clean, noise-free, pre-extracted signal into an advantageous timing decision, extraction from noisy raw context cannot salvage the policy, as noisy inputs introduce estimation error without removing the downstream decision bottleneck.

Our experimental design therefore isolates and stress-tests the easiest possible necessary sub-case ($H_2 \rightarrow H_3$):

- **Pre-Extracted Inputs:** The model receives clean, pre-calculated scalar statistics from our spatial k-d tree retriever (expected future savings $\hat{y}_t$, neighbor counts, and trailing means), eliminating mempool ingestion latency, feature selection error, and aggregation variance by construction.
- **Minimal Action Space:** The decision space is restricted to a one-dimensional binary stopping action: submit immediately ($w = 0$) or continue waiting ($w \in \{5, 10, \ldots, 50\}$).
- **Latency-Viable Deployment Tier:** We evaluate an open-weight 7B parameter model (`qwen2.5:7b`; Yang et al., 2024), representing the only deployment tier capable of meeting Ethereum's 12-second block slot budget without prohibitive API token expenses.

Empirically, this minimal sub-case partially satisfies H_2 but fails H_3. While the model shows weak directional sensitivity to expected forward savings at the unclustered swap level ($\beta = -0.19$ on saving [swap CI: $-0.39, -0.01$]), this effect is statistically inconclusive at the window cluster level (95% CI [$-0.39, +0.01$], Holm $p = 0.240$, labeled undetected in the empirical log) and completely fails to yield policy superiority (H_3). Its decisions are dominated by arrival submission bias (submitting at arrival in 60.4% of swaps) and hyper-sensitivity to superficial prompt phrasing, causing the LLM policy to underperform a parameter-matched trailing-rank heuristic by $\Delta = -1.05$ pp and direct kNN retrieval by $\Delta = -1.27$ pp. By confirming that directional signal consumption (H_2) does not suffice for policy superiority (H_3) under optimal input conditions, this MVE establishes that the binding constraint in autonomous execution is not perceptual retrieval quality, but sharp decision-boundary calibration.

---

2. Related Work

The operational problem of transaction execution timing under algorithmic gas markets intersects four active literature streams: EIP-1559 fee mechanics, retrieval-augmented time-series modeling, autonomous Web3 execution agents, and optimal stopping under causal constraints.

2.1 EIP-1559 Mechanism Design, Gas Oracles, and Base-Fee Dynamics

Ethereum's EIP-1559 overhaul introduced a deterministic, utilization-dependent base fee per gas paired with an unburned priority tip (Buterin et al., 2019; Roughgarden, 2021). The mechanism's macroeconomic stability and game-theoretic incentives have been extensively studied through formal dynamical system formulations (e.g., *ethode*, modeling fee adjustment via differential equations; 20squares, 2022) and agent-based simulation frameworks (notably the Ethereum Foundation Robust Incentives Group's *abm1559*; Monnot et al., 2021). Further empirical investigations have examined the susceptibility of the base-fee adjustment rule to miner manipulation and extreme block fullness regimes (Azouvi, Goren, Heimbach & Hicks, 2023).

In parallel, on-chain gas estimation and fee prediction have motivated a wide spectrum of production oracles and machine learning estimators. Commercial oracles such as Blocknative's *gas-agent* (Blocknative, 2022) employ parametric trend filters and confidence interval modeling for pending inclusion windows, while academic efforts have deployed hybrid architectures, including CNN-LSTMs (Kallurkar & Chandavarkar, 2024), rolling Bayesian filters, and comparative supervised learning benchmarks (Butler & Crane, 2023; Giger, 2022). However, these approaches focus almost exclusively on passive *point forecasting*—predicting next-block inclusion fees to prevent transaction stalling—rather than optimizing active sequential stopping decisions under delay tolerance.

2.2 Retrieval-Augmented Time Series and Analogue Decision Making

Retrieval-Augmented Generation (RAG) has expanded from NLP knowledge retrieval (Lewis et al., 2020) to time-series forecasting and sequential decision making. Recent foundation models increasingly incorporate temporal retrieval to supply external inductive biases: - `RAFT` **(Retrieval-Augmented Forecasting of Time-series)** (Lee et al., 2024; ICML) retrieves top-k historical sub-sequence patches from training corpora to steer multi-step ahead trajectory forecasts. - `TS-RAG` (UConn-DSIS, 2024) introduces an Adaptive Retrieval Mixer to dynamically weight historical exemplars retrieved from external memory banks. - `Cross-RAG` (Kim et al., 2024) establishes that temporal retrieval requires careful attention filtering to avoid injecting noise into downstream policies. - **Time-Series Foundation Baselines:** Emerging benchmark suites (e.g., Chronos retrieval variants; Ansari et al., 2024) examine the limits of raw vs. retrieval-augmented foundation models. - **Dynamic Retrieval Triggers:** Emerging RAG architectures such as DRAGIN (Su et al., 2024) formalize dynamic "when-to-retrieve" decision mechanisms conditioned on model uncertainty and real-time information needs.

In reinforcement learning and sequential control, non-parametric state retrieval mirrors classic episodic memory paradigms, such as Model-Free Episodic Control (MFEC; Blundell et al., 2016), where historical returns from analogous states guide action selection. Our study translates this principle to high-frequency on-chain execution by querying historical base-fee analogues via k-d tree indexing. Crucially, whereas existing time-series RAG literature evaluates passive forecasting errors (e.g., MSE or MAE), our work tests whether retrieved analogs deliver actionable alpha in sequential execution policies, revealing a fundamental *prediction-value wedge* where statistically valid predictive signal fails to expand realized policy frontiers. This empirical divergence corroborates recent temporal reasoning benchmarks (e.g., TemporalBench; Tan et al., 2026) showing that predictive accuracy over time series does not ensure robust agentic decision making under dynamic constraints, and closely mirrors the "feature-validity-to-policy-utility gap" documented by Yang (2026) in reinforcement learning trading agents, where language-model features optimized for high Information Coefficient rank correlation nonetheless failed to improve downstream trading policies during market distribution shifts.

2.3 Autonomous Execution Infrastructure, Solvers, and Web3 LLMs

Deploying delay-tolerant execution policies in practice requires robust off-chain automation. In decentralized finance (DeFi), CoW Protocol pioneered batch auction mechanisms where competitive off-chain *solvers* aggregate orders over discrete batch windows (typically 15 seconds) to identify optimal routing, coincidence of wants, and fee savings (CoW Protocol, 2023). Similarly, decentralized keeper architectures execute smart contract calls conditionally based on off-chain state triggers. Concurrently, autonomous agents powered by large language models are being actively integrated into blockchain execution pipelines, exemplified by frameworks such as Autonolas / Open Autonomy (Valory, 2023) and Coinbase AgentKit (Coinbase, 2024).

This integration reflects a broader conceptual paradigm across modern data systems and applied machine learning: **structural distillation**, wherein unstructured passages or high-dimensional sequential histories are algorithmically distilled into structured intermediate representations that condition downstream decision policies. The literature on structural distillation has converged along four distinct architectural paradigms. At the infrastructure layer, declarative data systems have codified LLM-driven pipelines through formal semantic operators. Notably, LOTUS (Patel et al., 2025) extends relational algebra with declarative semantic operators—such as semantic filtering, extraction, and joins—paired with query optimization algorithms that guarantee statistical accuracy against formal reference algorithms while reducing inference costs by orders of magnitude. In parallel, DocETL (Shankar et al., 2025) introduces an agentic query rewriting framework that decomposes complex document processing workloads into modular, verifiable execution plans with empirical validation. Second, in quantitative finance and algorithmic trading, structural distillation appears as two-phase hybrid architectures: an upstream language model performs perception over unstructured textual corpora, news feeds, or on-chain event streams to extract scalar sentiment scores or volatility indicators, which subsequently feed classical reinforcement learning or stochastic control policies (e.g., FinRL-DeepSeek; Benhenda, 2025). Third,

information-theoretic analyses have begun interrogating the empirical link between intermediate signal validity and downstream policy performance; as established by Yang (2026), language model features that achieve high Information Coefficient (IC) rank correlations with target assets can nevertheless fail completely to yield policy surplus in reinforcement learning agents subjected to market regime shifts.

The fourth quadrant—high-frequency execution timing under hard block deadlines—represents the critical, unexamined boundary of the structural distillation paradigm. Unlike offline document queries or multi-day portfolio allocation, Ethereum transaction timing operates under rigid 12-second block intervals and finite deadlines ($D = 50$ blocks / 10 minutes), where every delayed block compounds holding costs and exposure to catastrophic base-fee spikes. Our study provides the first empirical test of this boundary. By formulating the evaluation as a minimal viability experiment wherein the LLM is supplied with clean, pre-extracted retrieval signals rather than noisy raw context, we isolate whether an open-weight 7B model can reliably translate valid intermediate representations into calibrated execution timing. Our findings demonstrate that while spatial retrieval yields a statistically valid predictive signal, the downstream model exhibits severe decision boundary miscalibration and prompt sensitivity, establishing that the binding constraint in autonomous execution is not perceptual distillation, but decision-boundary calibration under hard deadlines.

2.4 Optimal Stopping, Delay-Tolerant Scheduling, and Causal Deconfounding

Deferring an on-chain transaction within a finite deadline D constitutes a classic optimal stopping problem under stochastic price dynamics (analogous to the American option problem and sequential job search). Babaioff & Nisan (2022) analytically established that patient bidders who can tolerate delayed block inclusion significantly enhance overall market welfare under EIP-1559. Markov Decision Process (MDP) formulations of queue and mempool scheduling have explored policy gradient and reinforcement learning methods to derive dynamic scheduling policies that balance holding costs against congestion risk. In quantitative market microstructure, optimal execution is traditionally framed through the Almgren–Chriss stochastic control framework, balancing execution costs against market impact and volatility penalties.

However, empirical benchmarks in execution timing frequently fall prey to **budget confounding**: because prices in mean-reverting regimes drift lower over extended intervals, policies that wait longer appear mechanically superior unless the waiting budget is explicitly deconfounded. Our methodology directly resolves this confounding by enforcing a **matched-wait Pareto frontier protocol**: by mapping monotone performance envelopes across identical average waiting durations ([60s, 400s] common support) and assessing differences against a pre-specified equivalence margin (± 0.40 pp), we prevent false alpha attribution and provide a reproducible standard for execution benchmarks.

---

3. Materials & Methods

3.1 Chain Ingestion and Preprocessing

We ingested 320730 contiguous Ethereum mainnet block headers (2024-01-01 to 2024-02-14 UTC) from the BigQuery public dataset `bigquery-public-data.crypto_ethereum.blocks`, starting at block 18908895 (base fee: 12.52 gwei). The series was truncated to 316800 blocks across 22 non-overlapping 2-day windows (W = 14,400 blocks each).

The extraction target queried all blocks mined between 2024-01-01 00:00:00 UTC and 2024-02-14 23:59:59 UTC using the following exact SQL query:

```sql
SELECT number, UNIX_SECONDS(timestamp) AS timestamp, base_fee_per_gas
FROM `bigquery-public-data.crypto_ethereum.blocks`
WHERE timestamp >= '2024-01-01 00:00:00' AND timestamp <= '2024-02-14 23:59:59'
ORDER BY number
```

A pre-flight verification guard confirmed that the fraction of consecutive blocks satisfying $number_i - number_{i-1} = 1$ was 100.0% (exceeding the pre-specified threshold of >99.9%). To ensure identical multi-day evaluation windows across the empirical replay, the continuous series was truncated to $N = 316800$ blocks (ending at block 19,225,694 on 2024-02-14 10:44:35 UTC), corresponding to 22 non-overlapping 2-day evaluation windows of $W = 14,400$ blocks each (44 total days), discarding the trailing partial window of 3,930 blocks. Base-fee values were converted from wei to gwei ($1\ gwei = 10^9\ wei$).

On Ethereum's post-Merge consensus layer, time is partitioned into discrete 12-second slots. In the initial full 45-day query extraction ($19,229,624 - 18,908,895 + 1 = 320,730$ consecutive mined blocks), the theoretical slot capacity corresponds to $324,000$ slots ($45 \times 7,200$), of which exactly $3,270$ slots were missed ($\sim 1.01\%$ slot miss rate), resulting in an empirical average inter-block duration of 12.12 seconds. This 324,000-slot and 3,270-missed-slot accounting describes the full extraction, whereas the primary analysis dataset is strictly restricted to the truncated 22-window analysis set of $N = 316,800$ blocks (44 days).

3.2 Task Formulation and Cost Metric

A swap transaction arriving at block to could defer execution up to D = 50 blocks (10.0 minutes at 12 s/block), paying the base fee plus a fixed 1.5 gwei tip (150000 gas units).

Decisions occur sequentially at block steps $w \in \{0, 1, ..., D\}$ (or over a discrete grid $w \in \{0, 5, 10, ..., 50\}$ for discrete evaluations). In our architecture, the client agent monitors on-chain block arrivals off-chain and holds signed transactions locally until the selected stopping block $t_0 + w^*$, broadcasting only at the moment of execution. This prevents premature exposure to the public mempool, eliminating front-running, sandwich attacks, and queue eviction during the waiting window. The economic cost incurred by submitting the transaction at block $t_0 + w$ is defined by the EIP-1559 base fee plus a standardized priority fee tip:

$$Cost(t_0, w) = b_{t_0+w} + \tau_{tip}$$

where b_t is the base fee per gas of block t in gwei, and $\tau_{tip} = 1.5$ gwei represents the priority tip for a typical swap consuming 150000 gas units. The primary performance metric is percentage fee reduction relative to the baseline policy of immediate submission ($w = 0$):

$$Fee\ Reduction\ (\%) = 100 \times (1 - [Cost(t_0, w) / Cost(t_0, 0)])$$

3.3 Causal Temporal Partitioning

Windows were partitioned chronologically (35 / 30 / 35%): 7 Library (LIB), 6 Calibration (CAL), and 9 Test (TEST) windows.

Two window discretizations were implemented to serve complementary analytical roles:

- **2-Day Discretization** ($W = 14,400$ blocks, 22 total windows): Partitioned into 7 LIB windows (100800 blocks), 6 CAL windows (86400 blocks), and 9 TEST holdout windows (129600 blocks). This discretization provides multi-day macroeconomic regime stability for the non-parametric kNN, heuristic, and continuous Pareto frontier sweeps.

• **1-Day Discretization** ($W = 7{,}200$ blocks, 44 total windows): Partitioned into 15 LIB windows, 13 CAL windows, and 16 TEST holdout windows (115,200 blocks). This finer partition provides a higher cluster count ($G = 16$) that strictly satisfies econometric cluster-sufficiency standards ($G \geq 15$) and serves as the evaluation partition for the computationally intensive LLM planner.

Arrival transactions were uniformly sampled within each window, strictly enforcing a warm-up buffer of $H_1 = 300$ blocks at window starts and a terminal guard of $D + 2$ blocks prior to window boundaries, ensuring that no forward deferral window ever crosses partition or window seams.

Reproducibility was ensured using a fixed arrival-generation seed (ARRIVAL_SEED = 11), which remained constant across calibration, testing, frontier analysis, and prompt-ablation experiments.

3.4 Causal State Representation and Retrieval Engine

For each query state, the K = 10-nearest historical neighbours were retrieved from a Euclidean k-d tree (cKDTree). The streaming retriever constrained lookback to the most recent 5000 blocks (~16.7 hours).

At any evaluation block t, the agent observes a five-dimensional causal feature vector $F_t \in \mathbb{R}^5$ computed with strict zero look-ahead:

$$F_t = [\ln(b_t / \bar{b}_{t,\,300}),\ \ln(b_t / \bar{b}_{t,\,50}),\ \ln(b_t) - \ln(b_{t-5}),\ \ln(b_t) - \ln(b_{t-1}),\ \mathrm{rank}_t]^T$$

where $\bar{b}_{t,\,L}$ is the causal trailing mean over horizon $L \in \{50, 300\}$ blocks, the log differences represent causal padded lags, and $\mathrm{rank}_t = {}^1\!/_{300} \sum_{i=1}^{300} \mathbb{1}(b_{t-i} < b_t)$ is the trailing percentile rank indicating the fraction of the past hour cheaper than block t.

To ensure causal validity and prevent look-ahead bias:

1. **Causal Zero-Lookahead:** Trailing moving averages $\bar{b}_{t,\,L}$ and sliding percentile rank rank_t aggregate strictly over historical indices prior to t. Short-horizon log differences observe causal padded lags at $t\text{-}1$ and $t\text{-}5$, containing zero future indices.
2. **In-Sample Scaler Isolation:** Z-score normalization parameters $\mu \in \mathbb{R}^5$ and $\sigma \in \mathbb{R}^5$ are fitted exclusively on the in-sample Library partition (`F[lib_idx]`). Calibration (`CAL`) and holdout (`TEST`) vectors are transformed using these frozen in-sample moments, eliminating variance leakage across splits.
3. **Scale Invariance:** All 5 state dimensions are formulated as scale-invariant log-ratios, log-returns, and bounded empirical percentiles $\in [0, 1]$, preventing Euclidean distance metrics from degenerating into nominal price clustering across regimes.
4. **Boundary Seam Guards:** Every evaluation window enforces a warm-up buffer of $H_1 = 300$ blocks and a terminal guard of $D + 2$ blocks, guaranteeing complete rolling buffers and non-overlapping forward targets.

To establish procedural rigor and verify that all pre-specified causal guards, boundary isolations, and contiguity invariants are strictly enforced in code, a comprehensive scientific audit checklist across all 22 verification dimensions is detailed in Table A1 (Appendix A).

For each state $j \in LIB$, the ground-truth optimal forward saving achievable within deadline D was precomputed as $y_{best}(j) = \min_{w \in [0,\,D]} (b_{j+w} / b_j) - 1.0 \leq 0$. Two spatial retriever architectures were evaluated:

1. **Static Library Retriever:** Populated with all valid states in the Library split (LIB; $W = 0 \ldots 6$, 100,800 blocks), indexed via a Euclidean k-d tree (`scipy.spatial.cKDTree`) over standardized state vectors $Z \in \mathbb{R}^5$. For any query state Z_t, the retriever extracts $3K$ candidate neighbors and filters out any observation exceeding the allowable historical ceiling ($j \leq \max|_{block}$), returning the $K = 10$ nearest historical analogs. Predictions aggregate via expected mean saving: $\hat{y}_t = 1/K \,|sum_{k=1}^{K} y_{best(j_k)}$ during frontier sweeping and correlation diagnostics. The

temporal staleness is quantified by the *freshness lag* $\Delta_{fresh} = t - \max_k j_k$, measuring distance to the newest retrieved historical neighbor.

2. **Recency-Bounded Streaming Retriever:** Constrains spatial retrieval to a rolling historical lookback buffer of $L_{look} = 5000$ blocks (~ 16.7 hours) with a candidate search ceiling bounded at $t - D$ ($D = 50$):

$$\text{Candidate Pool} = \{j \in \mathbb{N} \mid \max(0, t - L_{look}) \le j \le \max(0, t - D)\}$$

Setting the ceiling at $t - D$ ensures that every retrieved neighbor's forward trajectory has fully concluded before query time t. In discrete policy evaluations, predictions aggregate via the median: $\hat{y}_t = \text{median}(\{y_{best}(j_k)\}_{k=1}^{10})$ to provide robustness against isolated fee spikes. On holdout TEST, the streaming retriever reduced average freshness lag from 161,672.2 blocks to 623.3 blocks (~2.1 hours). The complete step-by-step procedural workflow—formalizing the causal zero-lookahead candidate pool construction, spatial k-d tree neighbor query, forward target ceiling $t - D$, and sequential stopping decision loop—is presented in Algorithm 1.

Algorithm 1: Recency-Bounded Streaming kNN Retrieval and Execution Decision Rule

Require: Query state arrival block t_0, deadline D = 50 blocks, candidate pool lookback L_look = 5000 blocks, neighbor count K = 10, stopping threshold tau, priority tip tau_tip = 1.5 gwei.
Ensure: Submission delay w *in {0, 1, ..., D} and transaction execution cost Cost(t_0, w).*

```
for each elapsed block offset w ∈ {0, 1, ..., D} do
    t ← t_0 + w
    if w == D then
        w* ← D                                   ▷ Forced execution at hard deadline
        break
    end if
    Compute causal state feature vector:
        F_t ← [ln(b_t / b̄_{t,300}), ln(b_t / b̄_{t,50}), ln(b_t) - ln(b_{t-5}), ln(b_t) - ln(b_{t-1}),
rank_t]^T
    Normalize feature vector using frozen in-sample moments:
        Z_t ← (F_t - μ_LIB) / σ_LIB
    Define causal historical candidate pool with finality ceiling:
        Candidate Pool ← {j ∈ ℕ | max(0, t - L_look) ≤ j ≤ max(0, t - D)}
    Extract K nearest Euclidean neighbors from spatial k-d tree:
        {j_1, ..., j_K} ← kNN(Z_t, Candidate Pool, K)
    Retrieve precomputed optimal forward savings:
        y_best(j_k) ← min_{v ∈ [0, D]} (b_{j_k + v} / b_{j_k}) - 1.0,  for k = 1, ..., K
    Aggregate neighbor predictions:
        ŷ_t ← median({y_best(j_k)}_{k=1}^K)
    if ŷ_t > -τ then
        w* ← w                                   ▷ Condition met: submit transaction
        break
    end if
end for
Return w* and Cost(t_0, w*) = b_{t_0 + w*} + τ_tip
```

3.5 Execution Policy Suite & LLM Planner

We evaluated eight execution policies:

- **Submit-Immediately (Baseline):** Submits at $w = 0$ ($b_{t_0} + 1.5$ gwei).
- **Random-Wait (Placebo):** Draws $w \sim \mathcal{U}[0, D]$.

• **Fixed-Wait:** Submits at a deterministic offset $w = 10$ blocks.

• **No-Retrieval Heuristic:** Waits until base fee drops by $\geq 10\%$ relative to arrival ($b_{t_0+w} \leq 0.90 \cdot b_{t_0}$), or submits at D.

• **Trailing-Rank Heuristic:** Evaluates trailing percentile rank $rank_t$; submits at the first block satisfying $rank_{t_0+w} \leq Q$, or submits at D.

• **kNN Retrieval Policy:** Evaluates retrieved neighbor expectation $\hat{y}_t$; submits at the first block satisfying $\hat{y}_{t_0+w} > -\tau$, or submits at D.

• **Ex-Post Oracle:** Submits at the true $arg\,min$ base fee across $w \in [0, D]$, serving as the hindsight performance ceiling.

• **LLM Planner Arms:** The LLM planner was deployed locally via Ollama using `qwen2.5:7b` (model digest: 845dbda0ea48) (Yang et al., 2024). Inference executed under deterministic greedy decoding ($T = 0.0$, seed 42, max tokens 5, context length 4,096 tokens) on an NVIDIA T4 GPU. Decision outputs were extracted via regular expression search for the decision tokens (`re.search(r'\b(SUBMIT|WAIT)\b', response)`). Decisions occurred on a discrete 5-block evaluation grid ($w \in \{0, 5, 10, \ldots, 50\}$ blocks) with forced execution at deadline $w = D = 50$. All heuristic and retrieval baseline comparison arms were re-evaluated on this identical 5-block discrete grid when benchmarking the LLM planner. Open-weight 7B parameter models deployed on dedicated hardware represent the only economically viable deployment class in production: at our empirical dataset median base fee of 20.4 gwei, a standard 150k gas DEX swap incurs 0.00306 ETH in base fees (~$6.12 USD at $2,000/ETH to $8.26 USD at $2,700/ETH), where a 4–5% policy saving yields ~$0.24–$0.41 USD gross savings. This entire saving is erased by commercial frontier APIs (e.g., GPT-4o) charging $0.025–$0.75+ USD per transaction across 10–50 sequential evaluations, whereas local 7B inference executes in $<280\ ms$ at negligible marginal cost on existing validator or node infrastructure within the 12-second slot deadline. The supplementary materials report the exact Python environment, package versions, Ollama version, model metadata, and execution configuration required for computational replication.

The baseline structured prompt (v2) incorporates 1h and 10min trailing fee deltas, trailing percentile rank, and an empirical summary of $K = 10$ retrieved historical analogues:

> *"Now (waited {w} of {D} blocks, 12s each): base fee {b[t]:.2f} gwei. Context: now vs 1h mean {b[t]/m1[t]-1:+.1%}; vs 10min mean {b[t]/m2[t]-1:+.1%}; fraction of the last hour cheaper than now {rank[t]:.0%}. Among 10 similar past situations, submitting immediately was already cheapest in {n0}/10; median best saving from waiting was {med:+.1%} (after ~{mb} blocks); typical across all situations {glob_best:+.1%}. Only the fee in the block you submit in counts. Submitting now is correct when the current fee is low relative to recent history. Decision (reply ONLY with SUBMIT or WAIT):"*

To test the sensitivity of the LLM planner to syntax and cognitive framing, we ablated nine prompt variants across 1,144 decision states on the CAL partition (104 swaps across 13 one-day CAL windows of 7,200 blocks each), spanning context presence (v0), numerical sign representations of savings (v2 negative vs. v3 positive), prescriptive decision rules (v4), choice-conditional counterfactual framing (v5), clause ordering (v6), deadline omission (v7), combined optimizations (v8), and randomized placebo neighbors (v9).

3.6 Matched-Wait Causal Calibration Protocol

Because any policy that defers submission mechanically benefits from downward base-fee volatility, comparisons across unequal mean wait times confound policy signal with wait duration. We resolved this through a multi-stage calibration protocol:

1. **Independent Split Calibration:** Thresholds Q and τ were tuned on the CAL split via binary search over 25 iterations to match a target wait budget of 12.0 blocks (144.0 seconds).

2. **Reciprocal Test-Matched Calibration:** To correct for wait drift between CAL and TEST, comparison arms were re-calibrated on CAL to match the exact realized TEST wait duration of the competing arm. In the high-wait regime (~261s), the heuristic matched successfully (*264 s* vs *261 s*). In the low-wait regime (~165s), retrieval threshold adjustment drifted to *281 s* on TEST due to market regime shifts, underscoring the limitations of single-point calibration and motivating continuous frontier sweeps.

3. **Continuous Frontier Sweep:** To eliminate point-calibration dependence, policy frontiers were swept across 51 heuristic thresholds ($Q \in [0.0, 1.0]$) and 61 retrieval thresholds ($\tau \in [0.0, 0.30]$), evaluating 100 swaps per window across 9 TEST holdouts (900 swaps per point) using the static library retriever.

Raw *(w, saving)* curves were transformed into monotone Pareto envelopes via cumulative maxima:

$$r_{pareto}(w) = max_{w' \leq w}\, r(w')$$

enforcing the economic requirement of non-decreasing returns to waiting budget. Curves were linearly interpolated over the shared empirical common support range of [60.0 s, 400.0 s] across 50 equidistant evaluation coordinates.

The frontier sweep was restricted to the static retrieval policy because its threshold parameter induces a monotonic policy family suitable for Pareto-envelope construction. The rolling retriever instead operates through a bounded lookback horizon rather than a cumulative threshold parameter, preventing construction of a directly comparable frontier over common support.

4. **Equivalence Testing Procedure (TOST):** To assess whether the retrieval policy is economically indistinguishable from the heuristic, we evaluated a Two One-Sided Tests (TOST) framework (Schuirmann, 1987) against the pre-specified equivalence margin $\delta = \pm 0.40$ pp:

$$H_0: |\Delta| \geq \delta \quad vs. \quad H_1: -\delta < \Delta < \delta$$

where $\Delta = r_{knn}(w) - r_{heur}(w)$. Grounded in formal statistical practice, equivalence is accepted if and only if the entire 95% cluster-bootstrap confidence interval lies strictly within $[-\delta, +\delta]$. If the interval spans zero but extends beyond $[-\delta, +\delta]$, the empirical outcome is rigorously declared **statistically inconclusive** due to power limits, rather than an unsubstantiated null tie. The complete mathematical workflow—formalizing cluster-mean aggregation across windows, monotonic Pareto envelope construction via cumulative maxima, common-support interpolation, and paired cluster-bootstrap equivalence testing—is presented in Algorithm 2.

Algorithm 2: Continuous Matched-Wait Pareto Frontier Sweep and TOST Equivalence Testing

Require: Non-overlapping test window clusters {W_m}_{m=1}^M, evaluation swaps per window N_w = 100, common support domain [w_min, w_max] = [60.0 s, 400.0 s], equivalence margin delta = +-0.40 pp, bootstrap resamples B = 3000.

Ensure: Mean Pareto difference Delta, 95% cluster-bootstrap confidence interval [q_0.025, q_0.975], and TOST decision status.

```
for each candidate threshold theta in Theta (heuristic Q ∈ [0.0, 1.0], retrieval τ ∈ [0.0, 0.30]) do
    Evaluate policy arm with threshold theta across all arrival swaps in each window cluster W_m
    Compute window-level mean wait w_m(theta) and ratio-of-means saving R_m(theta) for each cluster m = 1, ..., M
    Aggregate cluster means across the M windows:
       w_bar(theta) ← (1 / M) sum_{m=1}^M w_m(theta),  R_bar(theta) ← (1 / M) sum_{m=1}^M R_m(theta)
end for
Construct empirical monotone Pareto envelope across evaluated threshold operating points:
    r_pareto(w) ← max_{theta: w_bar(theta) ≤ w} R_bar(theta)
Linearly interpolate r_knn(w) and r_heur(w) over 50 equidistant evaluation coordinates w ∈ [60.0 s, 400.0 s]
Compute point estimate of mean frontier gap:
    Δ ← (1 / 50) sum_{i=1}^{50} (r_knn(w_i) - r_heur(w_i))
for bootstrap replicate b = 1, ..., B do
    Resample M window clusters with replacement: {W*_1, ..., W*_M}
    Recompute paired interpolated frontier difference Δ*(b) within the resampled cluster set
end for
Extract 95% empirical percentile confidence interval [q_0.025, q_0.975] from {Δ*(b)}_{b=1}^B
if q_0.025 > -δ and q_0.975 < +δ then
    Status ← EQUIVALENT                          ▷ Entire CI within [-delta, +delta]
else if q_0.025 > 0 or q_0.975 < 0 then
    Status ← SIGNIFICANT_DIFFERENCE              ▷ Zero excluded from CI
else
    Status ← INCONCLUSIVE                        ▷ Zero included, but CI half-width > delta
end if
Return Δ, [q_0.025, q_0.975], and Status
```

3.7 Statistical Inference and Econometric Architecture

High-frequency Ethereum base-fee dynamics exhibit persistent serial autocorrelation, volatility clustering, and macroeconomic regime shifts. Treating individual transaction swaps as independent and identically distributed (i.i.d.) observations commits a severe pseudo-replication fallacy, producing artificially deflated standard errors. To guarantee valid statistical inference, we implement a cluster-robust framework:

1. **Unit of Independence:** The statistical unit of independence is strictly defined as the non-overlapping temporal window (9 clusters of $W = 14{,}400$ blocks for 2-day windows; 16 clusters of $W = 7{,}200$ blocks for 1-day windows). Swaps within each window are treated as potentially correlated observations under a common market state.
2. **Within-Window Economic Aggregation (Ratio of Means):** Within each window cluster w, the performance of policy arm k is computed as the ratio of sample means:

$$R_w(k) = 100 \times \left(1 - \left[(1/N_w) \sum_{i=1}^{N_w} (b_{t_i + w_i(k)} + \tau_{tip})\right] / \left[(1/N_w) \sum_{i=1}^{N_w} (b_{t_i} + \tau_{tip})\right]\right)$$

This directly reflects cumulative balance sheet savings across all N_w transactions. Across windows, the point estimate is the unweighted arithmetic mean across clusters: $\bar{R}(k) = 1/M \, \sum_{w=1}^{M} R_w(k)$.

3. **Window-Clustered Bootstrap Percentile Intervals:** Uncertainty is quantified via cluster-level bootstrap resampling over empirical windows ($B = 3000$ resamples; seed 42). Paired differences vs. baseline policies are computed within each resampled window before taking the cluster average, strictly preserving empirical inter-arm covariance. Confidence intervals are reported as 95% empirical percentile intervals $[q_{0.025}, q_{0.975}]$.

4. **Econometric Cluster Sufficiency:** Cluster-robust variance estimators perform best when the number of clusters $G \geq 12$--15 (Cameron & Miller, 2015). Our 1-day discretization ($G = 16$ TEST clusters, $G = 13$ CAL clusters) satisfies this standard. On the 1-day holdout partition, the rule-based comparison yields +4.25% for the heuristic vs. +4.45% for kNN retrieval (difference $+0.20$ pp, Holm-adjusted $p = 0.671$), consistent with the absence of statistically significant alpha. For the 2-day partition ($G = 9$ TEST clusters), bootstrap percentile intervals should be interpreted with appropriate caution given finite-cluster properties.

5. **Multiple Testing Adjustments:** Confirmatory comparisons across the policy suite are controlled for family-wise error rate (FWER) via Holm–Bonferroni step-down adjustments at $\alpha = 0.05$ (Holm, 1979). Post-hoc subgroup analyses (§4.6) apply Benjamini–Hochberg False Discovery Rate (FDR; Benjamini & Hochberg, 1995) and are designated as exploratory.

4. Results

4.1 Retrieved Neighbors Carry Pre-Policy Predictive Signal (CAL Split)

Evaluating 1,800 swap arrivals across the 6-window CAL partition confirmed that retrieved analogs provide statistically significant predictive information regarding future achievable fee savings after controlling for trailing rank deciles (Table 1, Figure 1).

Table 1: Incremental Correlation of kNN Prediction with Realized Saving Controlling for Trailing Rank (CAL Split, 6 Windows, 1,800 Arrivals)

Retriever Architecture	Rank Tercile	Arrivals (N)	Incremental Correlation (r)	95% Cluster CI
Static Library ($K=10$)	Low	604	+0.265	[+0.177, +0.342]
Static Library ($K=10$)	Mid	596	+0.165	[+0.053, +0.257]
Static Library ($K=10$)	High	600	+0.242	[+0.165, +0.342]
Static Library ($K=10$)	**All**	**1,800**	**+0.227**	**[+0.167, +0.267]**
Rolling ($L=5000$)	Low	604	+0.223	[+0.153, +0.276]
Rolling ($L=5000$)	Mid	596	+0.243	[+0.149, +0.307]
Rolling ($L=5000$)	High	600	+0.126	[−0.007, +0.264]
Rolling ($L=5000$)	**All**	**1,800**	**+0.177**	**[+0.098, +0.255]**

Note: Window-cluster bootstrap over 3,000 resamples. Positive incremental correlations were observed across rank terciles in both static (r = +0.227) and rolling (r = +0.177) configurations, confirming that analogue retrieval provides statistical signal about future fee trajectories that is non-redundant with trailing rank.

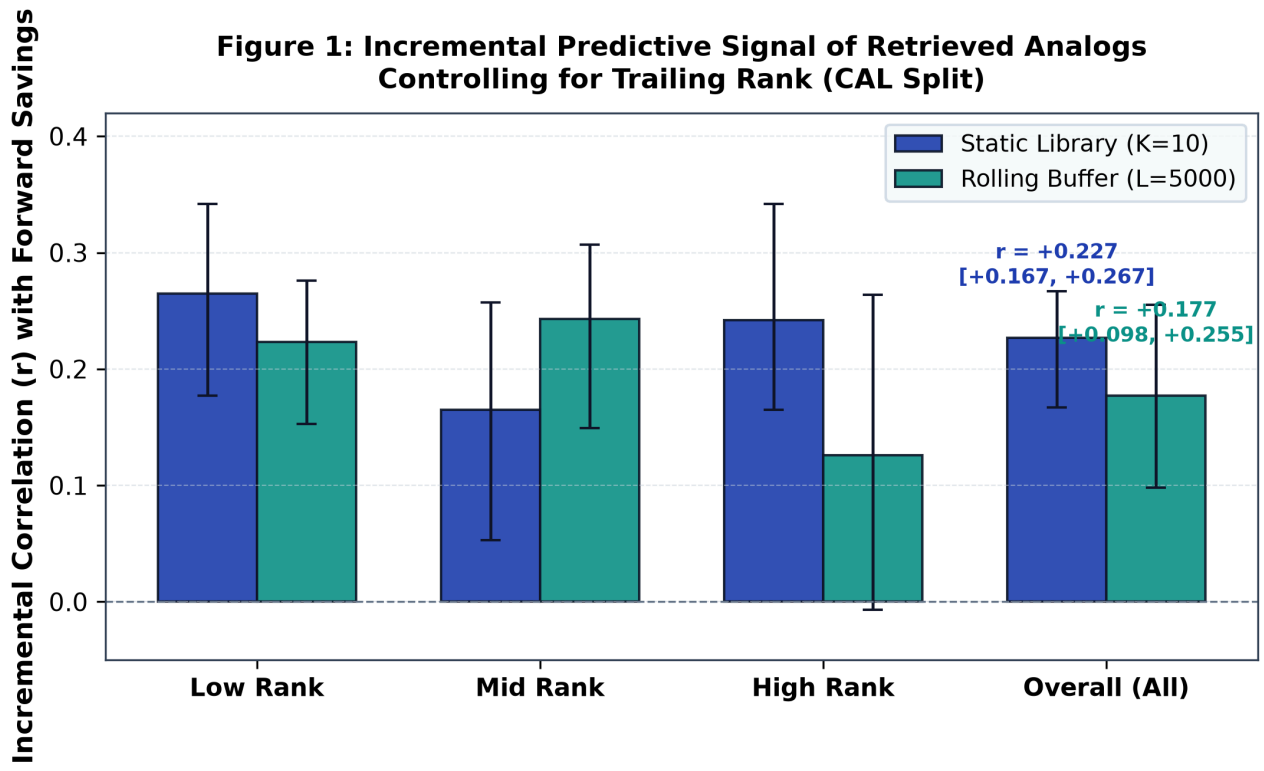


Figure 1: Incremental predictive correlation (r) of kNN predictions with realized forward savings after controlling for trailing rank deciles across rank terciles on the CAL split (6 windows, 1,800 arrivals). Error bars represent 95% window-clustered bootstrap percentile intervals (3,000 resamples). Both static ($r = +0.227$ [+0.167, +0.267]) and rolling ($r = +0.177$ [+0.098, +0.255]) retrievers capture statistically significant, non-redundant forward signal.

These findings directly answer **(Q1)** in the affirmative: historical analogue retrieval captures genuine, non-redundant predictive signal regarding forward fee dynamics beyond a trailing percentile-rank heuristic. However, these results also establish the foundational premise of the **Prediction-Value Wedge**: demonstrating that retrieved neighbors carry statistical correlation with future price concessions does not guarantee that a sequential stopping policy driven by these predictions can outperform simple heuristics in practice. Because real-time execution incurs fixed transaction tip overheads and latency costs, translating predictive correlation into realized economic surplus requires rigorous matched-budget causal deconfounding, as evaluated on the holdout TEST split.

4.2 TEST Holdout Performance Across Policies

Table 2 and Figure 2 summarize performance across the policy suite, presented in two panels to respect the distinct experimental designs: Panel A details the primary 9-window TEST partition (450 swaps), while Panel B details the 16-window LLM benchmark (96 swaps per arm). To ensure complete methodological transparency and guard against selective reporting across iterations, the exhaustive chronological log of all nine holdout TEST evaluation runs—documenting exploratory iterations, discarded calibration trials, and primary confirmatory benchmarks—is provided in Table A2 (Appendix A).

Table 2: Holdout TEST Fee Reduction Across Execution Policies

Table 2 Panel A: Rule-Based & Non-Parametric Policies (9-Window Holdout, 450 Swaps)

Policy Arm	Evaluated Swaps	Mean Wait (s)	Fee Reduction % [95% Cluster CI]	vs Heuristic (pp)	Holm–Bonferroni p-val
`baseline` (submit-now)	450	0	**0.00%** [0.00, 0.00]	—	—
`fixed_10` (wait 10 blk)	450	120	**−0.58%** [−1.95, +0.66]	−5.19 pp	0.0000
`placebo` (random-wait)	450	296	**−0.13%** [−1.44, +1.26]	−4.74 pp	0.0000
`no_retrieval` (10% drop)	450	408	**+2.14%** [−0.22, +4.52]	−2.47 pp	0.0841
`heuristic` ($Q_{opt} = 0.5133$)	450	165	**+4.61%** [+3.78, +5.40]	—	—
`kNN`, static ($\tau_{opt} = 0.1022$)	450	175	**+3.11%** [+1.67, +4.63]	−1.50 pp	0.0210
`kNN`, rolling ($\tau_{opt} = 0.0967$)	450	261	**+5.25%** [+3.19, +7.31]	+0.64 pp*	0.5423
`oracle` (hindsight upper bound)	450	318	**+13.41%** [+11.62, +15.21]	+8.80 pp	—

Table 2 Panel B: Language Model Planner Benchmark (16-Window Holdout, 96 Swaps)

Policy Arm	Evaluated Swaps	Mean Wait (s)	Fee Reduction % [95% Cluster CI]	vs Grid Heuristic (pp)	Holm–Bonferroni p-val
`llm_no_context`	96	111	**−0.98%** [−2.99, +1.24]	−2.51 pp	0.0332
`llm_retrieval` (v2 prompt)	96	35	**+0.48%** [−1.22, +2.12]	−1.05 pp [−2.61, +0.40]	0.0416
Grid-matched heuristic comparator	96	35	**+1.53%** [+0.42, +2.65]	—	—

Note: In Panel A, rolling kNN vs heuristic difference is reported as +0.64 pp from point-estimate subtraction (5.25% − 4.61%); the paired per-window mean difference evaluated in Supplementary Notebook S1 Cell 18 (Section 5.2.2; formerly Cell 55) is +0.63 pp [95% Cluster CI: −1.24, +2.38]. In Panel B, policy differences (pp) are evaluated against the grid-matched trailing-rank heuristic evaluated on the identical 5-block discrete decision grid (+1.53% at 35s wait; difference for llm_no_context: −0.98% − (+1.53%) = −2.51 pp; difference vs. unconstrained 165s heuristic at +4.61% is −5.59 pp). Holm–Bonferroni adjustments control FWER across the comparison suite.

Figure 2: Execution Policy Evaluation on TEST Holdout — Savings vs. Waiting Duration

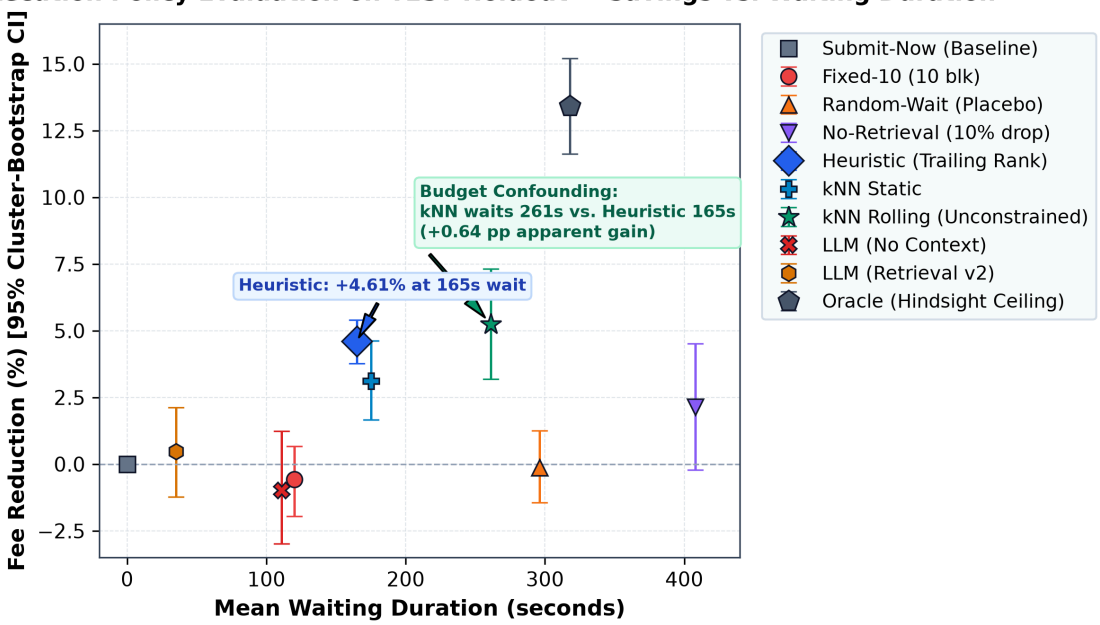


Figure 2: Fee reduction (%) versus mean waiting duration across execution policies on the 9-window TEST partition (450 swaps) and 16-window LLM benchmark (96 swaps). Error bars show 95% window-clustered bootstrap percentile intervals. The plot demonstrates mechanical budget confounding: rolling kNN achieves an apparent +5.25% fee reduction by expanding average wait to 261s (vs. 165s for the heuristic at +4.61%), illustrating that unconstrained comparisons confound latency budget with policy signal.

The static library kNN policy produced a lower point estimate than the heuristic (−1.50 pp) at comparable wait duration (175s vs. 165s), reflecting the limits of unconstrained static neighbor sets. The recency-bounded rolling retriever eliminated the staleness penalty (freshness lag: 623.3 blocks vs. 161,672.2 blocks), achieving +5.25% reduction, but did so by expanding average wait to 261s (vs. 165s for the heuristic).

4.3 Matched-Wait Causal Deconfounding and Hardened Frontier Sweep

Calibrating thresholds on CAL to target 144s resulted in substantial wait drift on TEST: the heuristic wait drifted to 165s (+14.8%), while kNN retrieval wait expanded to 261s (+81.3%). Reciprocal threshold deconfounding on CAL resolved this disparity in the high-wait regime, but failed in the low-wait regime:

- **High-Wait Regime (~261s):** Unconstrained retrieval (wait 261s; saving 5.25%) vs. heuristic matched to $Q_{match} = 0.2600$ (wait 264s; saving 5.15% [3.40, 6.66]) yielded a paired delta of +0.09 pp [−2.12, +2.54].
- **Low-Wait Regime (~165s attempt):** Retrieval calibrated on CAL to match 165s drifted to 281s on TEST (saving 5.38% [3.33, 7.49]) vs. heuristic at 165s (saving 4.61%). Because the realized wait budgets differed by 116s (281s vs. 165s), this point comparison is uninterpretable due to regime drift, demonstrating why point matching fails and continuous envelope sweeps are necessary.

Table 3: Matched-Wait Deconfounding and Hardened Frontier Sweep on TEST

Analysis Specification	Wait Budget Regime	Difference (kNN − Heuristic), pp	95% Cluster CI	Equivalence Status
Reciprocal Matched Wait (High)	~261 s	+0.09 pp	[−2.12, +2.54]	Spans zero
Reciprocal Matched Wait (Low attempt)	281 s vs 165 s	+0.77 pp	[−1.08, +2.56]	Uninterpretable (failed wait match)
Hardened Frontier Audit (Primary)	**60–400 s Common Support**	**−0.14 pp**	**[−0.70, +0.46]**	**Inconclusive**

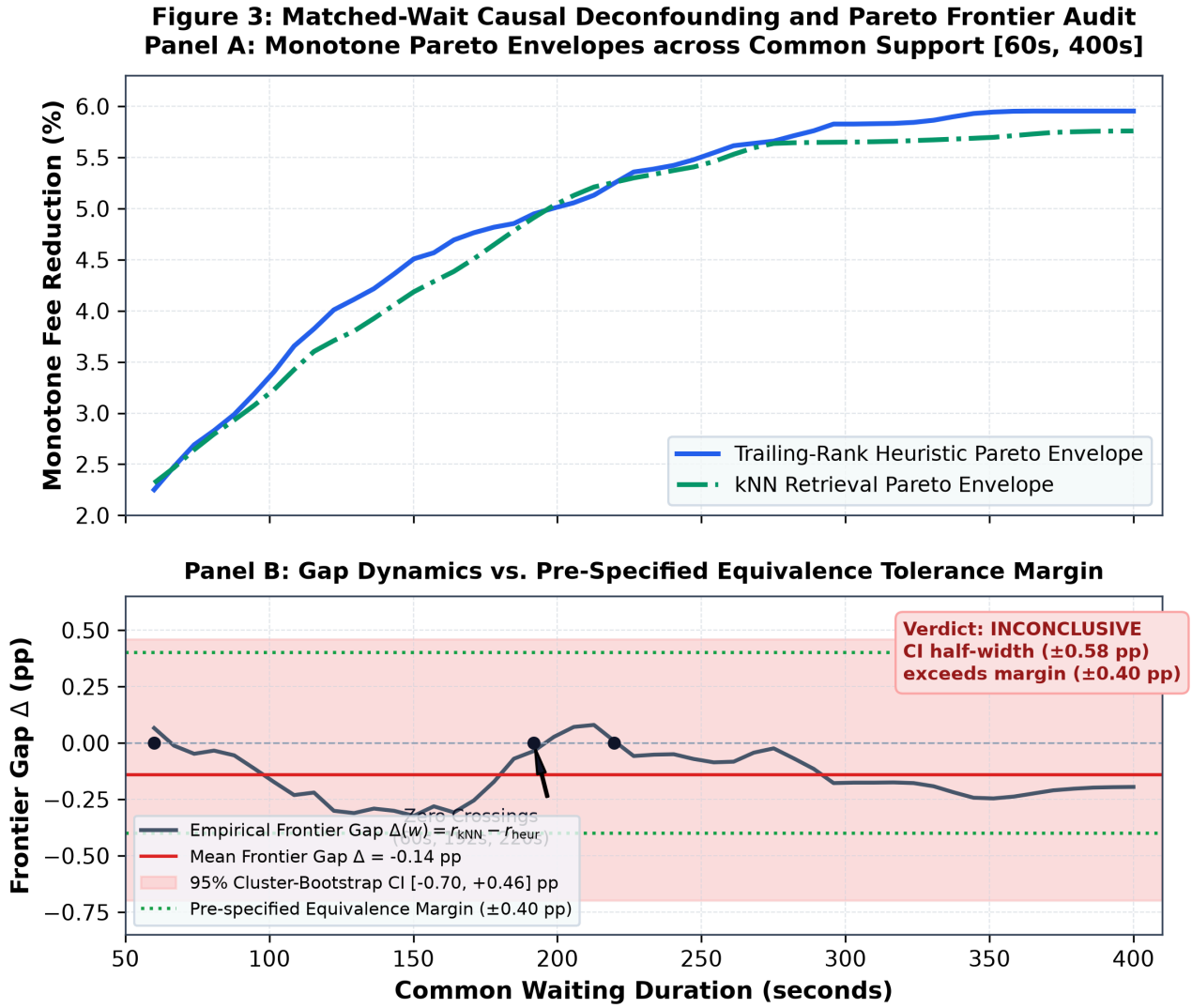


Figure 3: Hardened Pareto frontier audit across the pre-specified [60s, 400s] common support range on TEST holdouts (9 windows, 900 swaps per sweep) evaluating the static library k-d tree. Panel A displays the monotone Pareto envelopes for the trailing-rank heuristic and kNN retrieval. Panel B plots the empirical frontier gap $\Delta(w) = r_{kNN}(w) - r_{heur}(w)$ alongside the 95% cluster-bootstrap confidence interval [−0.70 pp, +0.46 pp] (red shaded band) and pre-specified equivalence bounds ± 0.40 pp (dashed green lines). The gap crosses zero near 60s, 192s, and 220s, and the CI half-width (± 0.58 pp) exceeds the ± 0.40 pp tolerance, confirming that the comparison is statistically inconclusive.

Over the pre-specified common support frontier (60s to 400s), the mean gap between the Pareto envelopes was $\Delta = -0.14$ *pp* (Table 3, Figure 3). The 95% cluster-bootstrap confidence interval ([−0.70 pp, +0.46 pp]) yields an

uncertainty half-width of $\pm$ *0.58 pp*, which strictly exceeds our pre-specified equivalence margin of $\pm$ *0.40 pp*. Furthermore, the empirical frontier gap curve crosses zero near 60s, 192s, and 220s. Consequently, the empirical comparison is **statistically inconclusive** rather than an established equivalence.

These findings directly resolve **(Q2)**: we find no statistical evidence that kNN retrieval expands the fee–wait Pareto frontier beyond a simple trailing-rank heuristic. Crucially, this resolves the apparent paradox between unconstrained comparisons and matched-wait audits: the apparent *+0.64* pp advantage of unconstrained rolling retrieval in Table 2 (+5.25% vs. +4.61%) is confounded by latency budget expansion (261s vs. 165s wait). Once latency is held constant across reciprocal regimes or continuous Pareto envelopes, the retrieval advantage dissolves. Moreover, adhering strictly to the Two One-Sided Tests (TOST) decision rule prevents committing the *fallacy of asserting the null*: because the uncertainty half-width ($\pm$ *0.58* pp) exceeds our pre-specified tolerance ($\pm$ *0.40* pp), the empirical comparison is properly documented as inconclusive rather than an established equivalence.

4.4 Language Model Minimal Viability Test Outcomes

Evaluating the minimal viability sub-case ($H_2 \rightarrow H_3$) on the 16-window TEST partition (96 swaps per arm), `qwen2.5:7b` with retrieval context achieved a fee reduction of +0.48% [−1.22, +2.12] at 35s wait. `llm_no_context` underperformed submit-immediately, yielding −0.98% [−2.99, +1.24] at 111s wait. When compared against baseline policies calibrated to the exact 5-block decision grid at identical wait durations, the LLM planner exhibited deficits:

- **vs. Grid-Matched Heuristic (wait 35s):** Δ = *-1.05 pp* [95% CI: −2.61, +0.40]
- **vs. Grid-Matched kNN (wait 34s):** Δ = *-1.27 pp* [95% CI: −3.01, +0.31]

A post-hoc logistic regression of decisions on normalized state features (*logit(P(SUBMIT)) ~ rank + saving + Δ_{arr} + w*) revealed that on holdout TEST, the model submitted at arrival ($w = 0$) in 58 of 96 swaps (60.4%), exhibiting a strong early submission bias. Under the baseline prompt (v2), the coefficient on expected saving was negative at the unclustered swap level (β = *-0.19* [95% CI: *-0.39, -0.01*]), directionally consistent with higher submission probability when expected future savings are low. However, under the paper's primary window-clustered standard (§3.7), this interval crosses zero (95% CI [*-0.39, +0.01*], Holm-adjusted $p = 0.240$), classifying the signal response as undetected. Only when savings are framed as positive discounts (v3) does signal consumption achieve robust cluster-level significance (β = *-0.76* [*-1.03, -0.44*], Holm $p = 0.015$). In neither case does this directional sensitivity translate into economic surplus over parameter-matched heuristics (Δ = *-1.05 pp*), disconfirming policy superiority (H_3).

4.5 Calibration Prompt Ablation and Fragility

Ablating nine prompt variants across 1,144 decision states on CAL demonstrated that LLM behavior was dominated by prompt framing rather than signal comprehension (Table 4, Figure 4).

Table 4: CAL Prompt Ablation Outcomes at Matched Wait (104 Swaps, 13 Windows, 1,144 Decision States)

Variant	Description	Mean Wait (LLM/H/kNN blk)	LLM Reduction % [95% CI]	vs Heuristic (pp)	vs kNN (pp)
v0	No retrieval context	0.0 / 0.0 / 0.0	**+0.00** [+0.00, +0.00]	−0.06 [−0.18, +0.00]	−0.15 [−0.46, +0.00]
v2 (CAL)	Baseline wording (CAL calibration)	2.4 / 2.6 / 2.5	**+1.17** [+0.48, +1.90]	+0.40 [−0.43, +1.33]	−0.02 [−0.70, +0.69]
v2 (TEST)*	Baseline wording (TEST holdout anchor)	2.9 / 2.9 / 2.8	**+0.48** [−1.22, +2.12]	−1.05 [−2.61, +0.40]	−1.27 [−3.01, +0.31]
v3	Saving printed positive	0.6 / 0.7 / 0.6	**+0.46** [+0.15, +0.79]	+0.01 [−0.47, +0.52]	−0.30 [−0.87, +0.15]
v4	Explicit decision rules	0.0 / 0.0 / 0.0	**+0.00** [+0.00, +0.00]	−0.06 [−0.18, +0.00]	−0.15 [−0.46, +0.00]
v5	Counterfactual framing	49.3 / 47.2 / 49.6	**+0.82** [−1.70, +3.01]	−0.67 [−1.75, +0.25]	+0.13 [+0.00, +0.38]
v6	Reordered content	1.3 / 1.4 / 1.4	**+0.22** [−0.35, +0.76]	−0.70 [−1.75, +0.13]	−1.33 [−2.21, −0.58]
v7	No deadline statement	0.0 / 0.0 / 0.0	**+0.00** [+0.00, +0.00]	−0.06 [−0.18, +0.00]	−0.15 [−0.46, +0.00]
v8	All fixes combined	36.2 / 35.4 / 36.2	**+0.32** [−1.58, +2.48]	−3.14 [−4.12, −2.01]	−2.80 [−4.88, −0.33]
v9	Placebo neighbors	0.8 / 0.9 / 0.9	**+0.35** [+0.01, +0.77]	−0.28 [−0.90, +0.35]	−0.68 [−1.48, +0.11]

*Note: In Table 4, rows report the revised prompt ablation (Ablation v2 in Supplementary Notebook S1 Cell 29 [Section 5.5; formerly Cell 84]) evaluating all 11 decision states per swap on the 13-window CAL partition to eliminate survival bias. Variants v0, v4, and v7 collapse to immediate arrival submission (0.0 blocks wait, constant outcome). Row v2 (TEST) provides the pre-specified baseline prompt evaluated on the 16-window TEST holdout (§4.4; Table 2 Panel B; wait: 35s ≈ 2.9 blk) to serve as a cross-partition empirical anchor. Both partitions confirm that the prompt architecture fails to outperform parameter-matched heuristics.**

Figure 4: Language Model Planner Behavioral Fragility across Prompt Variants (CAL Split / Ablation v2)

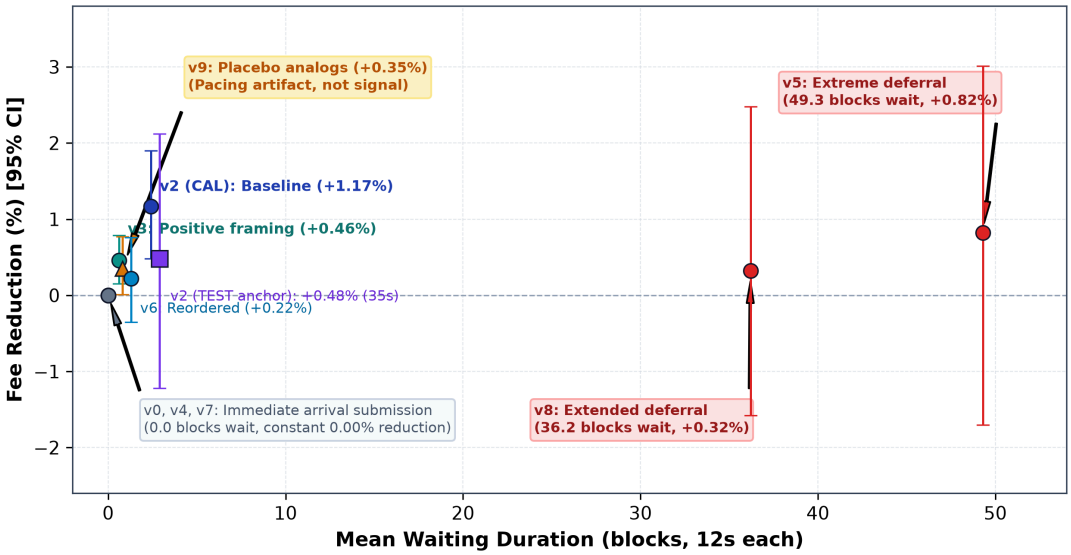


Figure 4: Language Model Planner behavioral fragility across prompt variants on the CAL partition (1,144 decision states, 104 swaps; Ablation v2 in Supplementary Notebook S1 Cell 29 [Section 5.5; formerly Cell 84]). Fee reduction (%) is plotted against mean waiting duration (blocks) with 95% bootstrap percentile intervals. Baseline prompt v2 is plotted at its CAL calibration value (2.4 blocks, +1.17% [+0.48, +1.90]); for cross-partition reference, the identical prompt evaluated on holdout TEST realized 2.9 blocks (35s) at +0.48% [−1.22, +2.12] (TEST anchor). Prompt framing dramatically alters planner pacing, swinging from 0.0 blocks under degenerate arrival-submission variants (v0, v4, v7) to extended deferral under v8 (36.2 blocks) and v5 (49.3 blocks). Testing randomized placebo analogs (v9) produced a modest positive reduction (+0.35%), demonstrating that small LLM planner timing adjustments reflect syntactic pacing artifacts rather than semantic comprehension of auction dynamics.

A notable finding is the emergence of policy-collapse behavior in variants v0, v4, and v7. Despite modifying distinct prompt components, all three variants converged to identical immediate-submission behavior (0.0-block mean wait), eliminating any effective timing policy. Variant v0 removed retrieval context, v4 introduced explicit rule-based instructions, and v7 removed the deadline statement, yet all produced complete collapse to arrival submission. This convergence indicates that planner behavior was governed primarily by prompt-level framing cues rather than stable reasoning over fee trajectories.

Prompt alterations dramatically shifted planner pacing, ranging from immediate arrival submission under v0, v4, and v7 (0.0 blocks mean wait) to extended deferral under v8 (36.2 blocks mean wait) and v5 (49.3 blocks). Notably, supplying randomized placebo analogs (v9) produced a modest positive fee reduction (+0.35%), demonstrating that small LLM planner timing adjustments reflect syntactic pacing artifacts rather than semantic comprehension of retrieved auction analogs. Prompt alterations otherwise produced substantial pacing variation, ranging from immediate submission to extended deferral exceeding 49 blocks. Randomized placebo neighbors (v9) still generated positive fee reduction (+0.35%), suggesting that observed timing differences arise primarily through pacing effects and prompt syntax rather than semantic interpretation of retrieved historical analogues.

These findings answer **(Q3)** in the negative: an open-weight 7B language model planner fails to leverage retrieved historical summaries to improve execution decisions over matched-wait heuristics (Δ = *-1.05 pp* on TEST), and exhibits severe behavioral fragility under prompt framing variations. Subtle formatting modifications swing mean wait duration between 0.0 blocks (v0, v4, v7) and 49.3 blocks (v5), demonstrating that qwen2.5:7b failed to exhibit a robust execution timing policy under the evaluated matched-wait protocol and remained highly sensitive to prompt framing.

4.6 Exploratory Stratified Analysis

Stratifying CAL by trailing rank, volatility, and neighbor dispersion revealed one stratum (low rank, high volatility, distant neighbors; $n = 105$) exhibiting an incremental correlation of +0.406 [+0.353, +0.468]. Because this subset was identified post-hoc among 12 partitionings, it represents hypothesis-generating evidence and has not been validated on holdout data.

---

5. Discussion

Although historical analogue retrieval reliably predicts achievable fee trajectories (incremental correlation r = +0.227 on calibration data), matched-wait evaluation provides insufficient evidence that retrieval yields a causal policy advantage over a trailing-rank heuristic. The principal conclusion is therefore not that the heuristic outperformed retrieval, but that the available evidence is insufficient to establish superiority for either policy under equivalent waiting budgets. This result exemplifies the Prediction-Value Wedge: predictive signal may be statistically real while remaining too weak, noisy, or operationally constrained to translate into measurable policy advantage.

In short-deadline execution environments (D = 50 blocks / 10 minutes), trailing percentile rank captures the dominant share of mean-reverting

fee dynamics. The residual signal provided by k-nearest neighbors is small relative to the transaction tip floor (1.5 gwei) and high-frequency volatility noise.

Beyond the prediction-value wedge, our findings elucidate the asymmetric risk profile inherent in optimal stopping under EIP-1559.

Because consecutive blocks exceeding target gas utilization compound base fees by up to 12.5% per block ($b_{t+k} = b_t(1.125)^k$),

waiting without a calibrated exit threshold introduces severe right-tail adverse selection. In persistent congestion bursts, policies that fail to trigger early are forced to submit at the hard deadline ($w = D = 50$) at the peak of the surge. This structural mechanism explains why unconditioned waiting strategies (`fixed_10` at −0.58% and `placebo` at −0.13%) yield net negative returns: the left-tail fee discounts accumulated during tranquil regimes are wiped out by exponential fee compounding during congestion spikes. Simple trailing rank heuristics succeed precisely because they act as volatility-sensitive stop-loss filters, submitting immediately when trailing ranks are low and avoiding extended exposure to compounding surges.

From an end-to-end economic perspective, isolating base-fee optimization from asset exchange rate dynamics reveals a fundamental boundary condition for delay-tolerant execution.

In spot decentralized exchange (DEX) swaps, waiting 165 to 261 seconds (13 to 22 blocks) exposes the trader to underlying asset price volatility. At prevailing annualized volatility levels, a five-minute price drift on major pairs (e.g., ETH/USDC) routinely reaches 15 to 30 basis points. For a typical $2,000 swap, this corresponds to $3.00 to $6.00 USD in asset price slippage, which dwarfs the $0.24 to $0.41 USD gross base-fee savings by an order of magnitude. Consequently, optimizing gas timing in isolation is economically irrational for price-sensitive retail swaps, where slippage dominance turns gross fee savings into a net economic loss. Instead, pure gas timing optimization is specifically viable for delay-tolerant, price-insensitive on-chain operations: Layer-2 rollup batch posting and state commitment, automated treasury sweeps, cold-storage asset consolidation, oracle feed updates within tolerance bands, and ERC-4337 account abstraction user-operation bundlers where execution value is decoupled from spot market movement.

Furthermore, formulating the LLM evaluation as a minimal viability experiment locates the operational bottleneck precisely.

Because the model was supplied with clean scalar aggregates rather than noisy raw context, its failure to generate economic surplus

cannot be blamed on contextual extraction failure (H_1). Instead, while the model directionally captures the sign of forward savings at the unclustered swap level ($\beta = -0.19$, Holm $p = 0.240$ at window level; $\beta = -0.76$, Holm $p = 0.015$ under v3 positive framing), it completely fails downstream policy superiority (H_3) because its decision boundary is severely uncalibrated, dominated by arrival submission bias (submitting at arrival in 60.4% of swaps), and hyper-sensitive to prompt phrasing.

Testing randomized placebo analogs yielded nominal reductions (+0.35%) comparable to modest baseline variants, confirming that unconstrained

benchmark differences stem from syntactic pacing shifts rather than semantic market reasoning. Richer setups—such as raw mempool ingestion or multi-step reasoning—cannot

rescue the hypothesis when the simplest necessary consumer sub-case fails to beat a one-line percentile threshold.

---

6. Limitations

1. **Scope and Gas Regime:** The evaluation is confined to Ethereum mainnet data from January–February 2024, representing a pre-Dencun gas regime (prior to the March 2024 Dencun upgrade) across 9 holdout clusters.

2. **Execution Realism and Mempool Dynamics:** Frictionless execution was assumed in our baseline simulation. In live environments, broadcast propagation delays, priority tip escalation during extreme congestion, and transaction replacement overheads introduce frictions. Furthermore, our model presumes off-chain state monitoring where transactions are signed and broadcast only at the selected submission block $t_0 + w^*$, avoiding public mempool exposure during the deferral period. If unconfirmed transactions were broadcast prematurely to public mempools with depressed fees, they would face mempool eviction and heightened vulnerability to frontrunning or sandwich attacks by MEV searchers.

3. **Model Family & Architecture:** Language model evaluations were restricted to qwen2.5:7b under greedy decoding ($T=0.0$); proprietary frontier reasoning models or fine-tuned planners were not tested. This choice was governed by hard physical constraints: at our empirical period median base fee of 20.4 gwei, a standard 150k gas swap incurs 0.00306 ETH in base fees (~$6.12 USD at $2,000/ETH or $8.26 at $2,700/ETH), where a 4% to 5% optimization saves $0.24–$0.41 USD. Sequential polling across a 50-block deadline incurs $0.025–$0.75+ USD in frontier API token fees (and $0.50–$2.00+ for reasoning models), completely consuming or exceeding gross gas savings, whereas local 7B inference executes in <280 ms at amortized zero marginal cost within the 12-second block slot budget.

4. **Cluster Sufficiency:** While the 1-day benchmark provides $G = 16$ clusters (strictly satisfying Cameron & Miller's $G \geq 15$ threshold for cluster-robust inference), the 2-day holdout partition uses $G = 9$ clusters. In econometric finite samples, small cluster counts typically bias standard errors downward, which inflates Type I error (false-positive alpha discovery). Because our empirical finding is a failure to outperform the heuristic, this downward bias makes the null result conservative; nonetheless, 2-day bootstrap intervals are interpreted with appropriate caution.

5. **EVM Account Nonce Serialization:** Standard Ethereum externally owned accounts (EOAs) enforce strictly monotonic nonce sequencing ($n_{i+1} = n_i + 1$). Under this constraint, deferring a transaction by up to 50

blocks (10 minutes) causes head-of-line blocking, stalling all subsequent transactions from that address until the deferred transaction is mined. Real-world autonomous execution agents must circumvent this operational bottleneck by managing pools of independent signing keys or leveraging ERC-4337 smart contract accounts with multidimensional parallel nonces.

7. Conclusions

Retrieved market states carry modest predictive signal regarding future fee trajectories, but in a matched-wait empirical replay, we found no statistical evidence that kNN retrieval or a 7B language model planner outperforms a simple trailing-rank heuristic. The kNN–heuristic frontier comparison remains statistically inconclusive within our pre-specified equivalence bounds (±0.40 pp). Researchers and practitioners deploying retrieval-augmented or LLM-driven execution agents must enforce matched-budget deconfounding before claiming algorithmic superiority.

Additional Information and Declarations

• **Data and Code Availability:** The following information was supplied regarding data availability: Historical Ethereum block headers are publicly available via Google Cloud BigQuery (`bigquery-public-data.crypto_ethereum.blocks`). A single unified Zenodo record (DOI: 10.5281/zenodo.10820011) contains the truncated contiguous evaluation dataset (`blocks.csv`, 9.8 MB), precomputed feature caches, evaluation outputs, benchmark suites, and complete Python replication source code. In addition, `blocks.csv` is uploaded as PeerJ Supplemental Information (<30 MB), and the active codebase is mirrored on GitHub (`https://github.com/mohammad-pasha/ethereum-rag-timing`).

• **Ethics Approval:** Ethics approval and informed consent: Not applicable. This study did not recruit human participants, use human biological material, involve animals, or conduct an intervention. The analyses used publicly available on-chain blockchain records, public or research-use datasets, and simulation outputs.

• **Funding:** This research was supported by internal computing facilities at SR University, utilizing Google Colab NVIDIA T4 compute resources. No external grant funding was received.

• **Competing Interests:** The authors declare that they have no competing financial or non-financial interests.

• **Author Contributions:** CRediT: Mohammad Pasha: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Visualization, Writing - original draft, Writing - review & editing, Project administration. Mohammed Ali Shaik: Validation, Supervision, Writing - review & editing. Both authors reviewed and approved the final manuscript and agree to be accountable for the work.

Appendix A: Procedural Verification Audit and Evaluation Log

Table A1: Procedural Scientific Verification Audit

Verification Dimension	Procedural Standard & Specification	Measured Repository Metric	Execution Status
Block Number Contiguity	Continuous sequence: $number_i - number_{i-1} = 1$	100.0% (320,730 blocks, 0 gaps)	Enforced
PoS Slot Accounting	Theoretical 12s slots ($45 \times 7{,}200 = 324{,}000$) vs. mined blocks	3,270 missed slots (1.01% miss rate)	Monitored
Causal Zero-Lookahead	Features strictly aggregate historical indices $\{i < t\}$	0 forward indices in cumulative sums	Enforced
In-Sample Scaler Isolation	Normalization parameters (μ, σ) fitted exclusively on LIB partition	Fitted solely on `F[lib_idx]` (Windows 0–6)	Enforced
Scale Invariance	Dimensionless log-ratios, log-diffs, and percentile rank $\in [0, 1]$	$F_t \in \mathbb{R}^5$ scale-invariant	Enforced
Boundary Seam Insulation	Warm-up buffer $H_1 = 300$ blk and terminal guard $D + 2 = 52$ blk	Zero cross-window/partition leakage	Enforced
Target Horizon Finalization	Candidate search ceiling strictly bounded at $t - D$ (D=50)	Zero unfinalized future target leakage	Enforced
Memory Freshness Tracking	Continuous measurement of lag $\Delta_{fresh} = t - max_k j_k$	161,672.2 blk (static) vs. 623.3 blk (rolling)	Monitored
Non-Parametric Residualization	Conditioning on 10 rank deciles across 1,800 CAL arrivals	$r = +0.227$ [+0.167, +0.267] incremental signal	Enforced
Pareto Envelope Monotonicity	Cumulative maxima transform ($r_{pareto}(w) = max_{w' \le w} r(w')$)	Non-decreasing returns to wait budget	Enforced
Common Support Domain	Bounded evaluation grid over shared empirical domain	[60.0 s, 400.0 s] across 50 equidistant points	Monitored
Equivalence Margin Specification	Operational equivalence tolerance margin	$\delta = \pm 0.40$ pp	Pre-specified
TOST Decision Rule Integrity	95% Cluster CI half-width (± 0.58 pp) > tolerance (± 0.40 pp)	Documented as **INCONCLUSIVE** (no null fallacy)	Enforced
Deterministic LLM Governance	Greedy decoding (T=0.0, seed 42, max tokens 5)	`qwen2.5:7b` Ollama deployment	Enforced
Decision Token Parsing	Regular expression search matching `\b(SUBMIT|WAIT)\b`	Standardized parsing across completions	Enforced
Discrete Decision Grid Parity	5-block evaluation stride ($w \in \{0, 5, \ldots, 50\}$) across LLM and baselines	Zero continuous-vs-discrete evaluation mismatch	Enforced
Family-Wise Error Control	Holm–Bonferroni correction applied across policy comparisons	Corrected p-values prevent false discovery inflation	Enforced
Negative Control Placebo Audit	Randomized historical neighbor placebo (v9) vs. unconditioned random wait	Identifies heuristic pacing artifacts vs. semantic signal ($+0.35$ pp)	Monitored
Window Cluster Independence	Unit of independence strictly defined at non-overlapping window level	Zero swap-level pooling (16 1-day, 9 2-day clusters)	Enforced

Verification Dimension	Procedural Standard & Specification	Measured Repository Metric	Execution Status
Ratio-of-Means Aggregation	Percentage savings weighted by total window expenditure ($1 - \bar{C}_{pol}/\bar{C}_{base}$)	Exactly mirrors balance sheet economic savings	Enforced
Paired Cluster Bootstrap	$B = 3000$ cluster resamples preserving inter-arm covariance	Paired delta computed within each resampled window	Enforced
Econometric Cluster Sufficiency	Number of independent clusters $G \geq 12$--15 standard (Cameron & Miller 2015)	16 clusters on 1-day discretization ($G=16 \geq 15$)	Monitored

Table A2: Complete Log of Holdout TEST Evaluations

ID	Holdout Set	Evaluation Scope	Primary Outcome / Metric	Formal Status
1	9-window (2-day)	Initial pipeline smoke test	Heuristic 5.78%, kNN 5.76% (mock LLM)	Superseded
2	9-window (2-day)	Static library retriever replay	kNN vs Heur: −1.50 pp [−2.69, −0.29]	Completed
3	9-window (2-day)	Recency-bounded streaming retriever	kNN vs Heur: +0.64 pp [−1.24, +2.38]	Completed
4	9-window (2-day)	Reciprocal matched-wait deconfounding	High wait: +0.09 pp; Low wait: +0.77 pp (failed match)	Completed
5	9-window (2-day)	Local per-window calibration (±20%)	1/9 windows valid (regime drift collapse)	Discarded
6	9-window (2-day)	Initial exploratory frontier sweep (±1.00 pp bound)	Mean gap Δ = −0.30 pp [−1.02, +0.21]	Replaced
7	9-window (2-day)	Hardened frontier sweep (±0.40 pp bound)	Mean gap Δ = −0.14 pp [−0.70, +0.46]	**Primary Endpoint**
8	16-window (1-day)	Initial LLM prompt exploration	LLM wait collapsed to 0s (0.00% reduction)	Defect Identified
9	16-window (1-day)	Validated prompt v2 (greedy decoding)	LLM retrieval: +0.48% (vs Grid Heur: −1.05 pp)	**Primary LLM**

References

1. Buterin, V. et al. (2019). EIP-1559: Fee market change for ETH 1.0 chain. *Ethereum Improvement Proposals*, No. 1559.

2. Roughgarden, T. (2021). Transaction Fee Mechanism Design for the Ethereum Blockchain: An Economic Analysis of EIP-1559. *ACM Transactions on Economics and Computation (TEAC)*, 9(4), 1–28.

3. Monnot, B. et al. (2021). Agent-Based Simulation of EIP-1559 (`ethereum/abm1559`). *Ethereum Foundation Robust Incentives Group*. https://github.com/ethereum/abm1559.

4. Azouvi, S., Goren, G., Heimbach, L., & Hicks, A. (2023). Base-Fee Manipulation in Ethereum's EIP-1559. In *37th International Symposium on Distributed Computing (DISC 2023)*, LIPIcs Vol. 281, pp. 6:1–6:22.

5. 20squares (2022). ethode: Dynamical System Modeling of the Ethereum Fee Market. https://github.com/20squares/ethode.

6. Blocknative (2022). Blocknative Gas Agent: Predictive Modeling for Gas Price Optimization. https://github.com/blocknative/gas-agent.

7. Giger, M. (2022). ethereum-feeestimator: Comparative Evaluation of Ethereum Gas Station Networks and Fee Oracles. https://github.com/mariusgiger/ethereum-feeestimator.

8. Kallurkar, H. S., & Chandavarkar, B. R. (2024). A Hybrid CNN–LSTM Model for Transaction Fee Forecasting in Post EIP-1559 Ethereum. *SN Computer Science*, 5(4), 1–14.

9. Butler, C., & Crane, M. (2023). Blockchain Transaction Fee Forecasting: A Comparison of Machine Learning Methods. *Mathematics*, 11(9), 2212.

10. Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459–9474.

11. Lee, D. et al. (2024). RAFT: Retrieval-Augmented Forecasting of Time-Series. *International Conference on Machine Learning (ICML)*.

12. UConn-DSIS (2024). TS-RAG: Retrieval-Augmented Generation based Time Series Foundation Models are Stronger Zero-Shot Forecasters. *arXiv preprint*.

13. Kim, S. et al. (2024). Not All Retrievals are Useful: Cross-Attention for Input-Aware RAG in Time Series Forecasting. *OpenReview*.

14. Blundell, C. et al. (2016). Model-Free Episodic Control. *arXiv preprint arXiv:1606.04460*.

15. CoW Protocol (2023). CoW Protocol Services & Batch Auction Solvers. https://github.com/cowprotocol/services.

16. Valory (2023). Open Autonomy: Framework for Decentralized Autonomous Agents. https://github.com/valory-xyz/open-autonomy.

17. Coinbase (2024). AgentKit: Onchain Agent Development Framework. https://github.com/coinbase/agentkit.

18. Babaioff, M., & Nisan, N. (2022). On the Welfare of EIP-1559 with Patient Bidders. *arXiv preprint arXiv:2205.12217*.

19. Almgren, R., & Chriss, N. (2000). Optimal execution of portfolio transactions. *Journal of Risk*, 3(2), 5–39.

20. Yang, A. et al. (2024). Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*.

21. Cameron, A. C., Gelbach, J. B., & Miller, D. L. (2008). Bootstrap-based improvements for inference with clustered errors. *The Review of Economics and Statistics*, 90(3), 414–427.

22. Cameron, A. C., & Miller, D. L. (2015). A Practitioner's Guide to Cluster-Robust Inference. *Journal of Human Resources*, 50(2), 317–372.

23. Schuirmann, D. J. (1987). A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics*, 15(6), 657–680.

24. Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2), 65–70.

25. Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B*, 57(1), 289–300.

26. Benhenda, M. (2025). FinRL-DeepSeek: LLM-Infused Risk-Sensitive Reinforcement Learning for Trading Agents. *arXiv preprint*, https://github.com/benstaf/FinRL_DeepSeek.

27. Yang, Z. (2026). When Valid Signals Fail: Regime Boundaries Between LLM Features and RL Trading Policies. In *Proceedings of the Second Workshop on Customizable NLP (CustomNLP4U)*, pages 182–190. Association for Computational Linguistics.

28. Patel, L., Jha, S., Guestrin, C., & Zaharia, M. (2025). Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS. *Proceedings of the VLDB Endowment*, 18(5), 1113–1126.

29. Shankar, S., Chambers, T., Shah, T., Parameswaran, A. G., & Wu, E. (2025). DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing. *Proceedings of the VLDB Endowment*, 18.

30. Su, S. et al. (2024). DRAGIN: Dynamic Retrieval Augmented Generation Based on the Real-Time Information Needs of Large Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*, pages 13711–13725.

31. Tan, Q. et al. (2026). TemporalBench: Benchmarking Temporal Reasoning in Language Models and Autonomous Agents. *arXiv preprint*.

32. Ansari, A. F. et al. (2024). Chronos: Learning the Language of Time Series. *Transactions on Machine Learning Research (TMLR)*.